HKBK COLLEGE OF ENGINEERING



Department of Electronics & Communication Engineering

B.E. in ECE

CS (IPCC) Lab Workbook

2022 Scheme AY 2025-26

Semester-4 | Course Code: BEC403

CONTROL SYSTEMS (BEC403)

IPCC LAB WORKBOOK

IV SEMESTER

2022 SCHEME AY 2025-26

Prepared by:
Prof. G Hitesh Reddy
Assistant Professor
Dept. of ECE

Reviewed by:
Dr. Hashinur Islam
Assistant Professor
Dept. of ECE

Approved by:
Dr Manjunath R Kounte
Dean – Academics, Professor & HOD
Dept. of ECE



Department of Electronics and Communication Engineering

HKBK College of Engineering

(Approved by AICTE, Affiliated to VTU, Belagavi)
S. No. 22/1, Nagawara, Bengaluru-560045, Karnataka, India
http://www.hkbk.edu.in Email: info@hkbk.edu.in
Phone: +91 80 25441722/ 3744/ 3690/ 3698 Fax: +91 80 25443813

HKBK College of Engineering

Department of Electronics and Communication Engineering

LAB WORKBOOK

2022 SCHEME AY 2025-26

CONTROL SYSTEMS

LABORATORY (IPCC)

BEC403

Name: _____

USN: _____

Batch: _____

Semester: _____ **Section:** _____

Faculty In-Charge

HKBK College of Engineering

S. No. 22/1, Nagawara, Opp. Manyata Tech Park, Bengaluru – 560 045.

Department of Electronics and Communication Engineering

Certificate

This is to certify that Mr./Ms. _____ has satisfactorily completed the **Control Systems (BEC403) IPCC** Laboratory as prescribed by the Visvesvaraya Technological University for the **IV** Semester B.E. (**Electronics & Communication Engineering**) during the academic year 20____ – 20____.

USN No.: _____

Batch: _____

Faculty Sign

Head of the Department

Date: _____ Principal

INDEX/SCORE SHEET

Sl. No.	Name of Experiment	Date of Completion	Page No.	Marks Obtained	Faculty Signature	Remarks
1	Implement Block diagram reduction technique to obtain transfer function a control system.					
2	Implement Signal Flow graph to obtain transfer function a control system.					
3	Simulation of poles and zeros of a transfer function.					
4	Implement time response specification of a second order Under damped System, for different damping factors.					
5	Implement frequency response of a second order System.					
6	Implement frequency response of a lead lag compensator.					
7	Analyze the stability of the given system using Routh stability criterion.					
8	Analyze the stability of the given system using Root locus.					
9	Analyze the stability of the given system using Bode plots.					
10	Analyze the stability of the given system using Nyquist plot.					
11	Obtain the time response from state model of a system.					
12	Implement PI and PD Controllers.					
13	Implement a PID Controller and hence realize an Error Detector.					

Faculty Signature with Date

VISION OF THE INSTITUTION

To empower students through wholesome education and enable the students to develop into highly qualified and trained professionals with ethics and emerge as responsible citizens with broad outlook to build a vibrant nation.

MISSION OF THE INSTITUTION

1. To achieve academic excellence through in-depth knowledge in science, engineering and technology through dedication to duty, innovation in teaching and faith in human values.
2. To enable our students to develop into outstanding professionals with high ethical standards to face the challenges of the 21st century.
3. To provide educational opportunities to the deprived and weaker section of the society, to uplift their socio-economic status.

VISION OF THE DEPARTMENT

To shape the students as disciplined humane engineers who can build a strong, peaceful and vibrant country and focus on mutual respect, tolerance and professional ethics.

MISSION OF THE DEPARTMENT

1. To provide the best possible educational experience.
2. Excellence in teaching and research activities for today's students and professionals of tomorrow.
3. To hone young minds and train them to be conscientious individuals.
4. Will serve the society as competent professionals in the field of Electronics and Communication Engineering.

Program Education Outcomes (PEOs)

PEO1: Our graduates will possess good knowledge in engineering fundamentals.

PEO2: Our graduates will be capable of analyzing and designing systems.

PEO3: Our graduates will be capable of creating innovative products in multidisciplinary areas.

PEO4: The graduates will be ethically strong personnel with good communication and interpersonal skills with high moral values.

PEO5: They will have a desire for continuous learning and Research and Development (R&D).

Program Specific Outcomes (PSOs)

PSO1: Professional Skills: An ability to understand the basic concepts in Electronics & Communication Engineering and to apply them to various areas, like Electronics, Communications, Signal processing, VLSI, Embedded systems etc., in the design and implementation of complex systems.

PSO2: Problem-Solving Skills: An ability to solve complex Electronics and communication Engineering problems, using latest hardware and software tools, along with analytical skills to arrive cost effective and appropriate solutions.

PSO3: An ability to become an entrepreneur or to contribute to industrial services and / or Govt. organizations in the field of Electronics and Communication Engineering.

PSO4: An ability to work on multidisciplinary teams with efficiency in different Programming techniques.

Program Outcomes (POs)

PO1: Engineering Knowledge: Apply the knowledge of Mathematics, Science, fundamentals and an engineering specialization to the solution of complex engineering problems.

PO2: Problem analysis: Identify, formulate, review research literature, and analyze complex Engineering problems reaching substantiated conclusions using first principles of mathematics, Natural sciences and engineering sciences.

PO3: Design/development of solutions: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.

PO4: Conduct investigations of complex problems: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the Information to provide valid conclusions.

PO5: Modern tool usage: Create, select, and apply appropriate techniques, resources, and modern Engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.

PO6: The engineer and society: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.

PO7: Environment and sustainability: Understand the impact of the professional engineering solutions of Electronics and Communication Engineering in societal and environmental contexts, and demonstrate the knowledge and need for sustainable development.

PO8: Ethics: Apply ethical principles and commit to professional ethics, responsibilities and norms of the engineering practice.

PO9: Individual and team work: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.

PO10: Communication: Communicate effectively on complex engineering activities with the engineering community and such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.

PO11: Project management and finance: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

PO12: Life-Long Learning: Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

CONTROL SYSTEMS		Semester	4
Course Code	**BEC403**	CIE Marks	50
Teaching Hours/Week (L:T:P: S)	3:0:2:0	SEE Marks	50
Total Hours of Pedagogy	40 hours Theory + 12 Lab slots	Total Marks	100
Credits	04	Exam Hours	03
Examination nature (SEE)	Theory/practical/Viva-Voce /Term-work/Others		

Course Objectives:

This course will enable students to

1. Understand basics of control systems and design mathematical models using block diagram reduction, SFG, etc.
2. Understand Time domain and Frequency domain analysis.
3. Analyze the stability of a system from the transfer function
4. Familiarize with the State Space Model of the system.

Teaching-Learning Process (General Instructions)

These are sample Strategies, which teacher can use to accelerate the attainment of the various course outcomes.

1. Lecture method (L) does not mean only traditional lecture method, but different type of teaching methods may be adopted to develop the outcomes.
2. Show Video/animation films to explain the different concepts of Linear Algebra & Signal Processing. • Encourage collaborative (Group) Learning in the class.
3. Ask at least three HOTS (Higher order Thinking) questions in the class, which promotes critical thinking.
4. Adopt Problem Based Learning (PBL), which fosters students' Analytical skills, develop thinking skills such as the ability to evaluate, generalize, and analyze information rather than simply recall it.
5. Topics will be introduced in a multiple representation.
6. Show the different ways to solve the same problem and encourage the students to come up with their own creative ways to solve them.
7. Discuss how every concept can be applied to the real world - and when that's possible, it helps improve the students' understanding.
8. Adopt Flipped class technique by sharing the materials / Sample Videos prior to the class and have discussions on the that topic in the succeeding classes.
9. Give Programming Assignments.

PRACTICAL COMPONENT OF IPCC

Using suitable simulation software (P-Spice/ MATLAB / Python / Scilab / OCTAVE / LabVIEW) demonstrate the operation of the following circuits:

Sl. No.	Experiments
1	Implement Block diagram reduction technique to obtain transfer function a control system.
2	Implement Signal Flow graph to obtain transfer function a control system.
3	Simulation of poles and zeros of a transfer function.
4	Implement time response specification of a second order Under damped System, for different damping factors.
5	Implement frequency response of a second order System.
6	Implement frequency response of a lead lag compensator.
7	Analyze the stability of the given system using Routh stability criterion.
8	Analyze the stability of the given system using Root locus.
9	Analyze the stability of the given system using Bode plots.
10	Analyze the stability of the given system using Nyquist plot.
11	Obtain the time response from state model of a system.
12	Implement PI and PD Controllers.
13	Implement a PID Controller and hence realize an Error Detector.
14	Demonstrate the effect of PI, PD and PID controller on the system response.

Course outcomes (Course Skill Set):

At the end of the course, the student will be able to:

1. Deduce transfer function of a given physical system, from differential equation representation or Block Diagram representation and SFG representation.
2. Calculate time response specifications and analyse the stability of the system.
3. Draw and analyse the effect of gain on system behaviour using root loci.
4. Perform frequency response Analysis and find the stability of the system.
5. Represent State model of the system and find the time response of the system.

CIE for the practical component of the IPCC

- **15 marks** for the conduction of the experiment and preparation of laboratory record, and **10 marks** for the test to be conducted after the completion of all the laboratory sessions.
- On completion of every experiment/program in the laboratory, the students shall be evaluated including viva-voce and marks shall be awarded on the same day.
- The CIE marks awarded in the case of the Practical component shall be based on the continuous evaluation of the laboratory report. Each experiment report can be evaluated for

10 marks. Marks of all experiments' write-ups are added and scaled down to **15 marks**.

- The laboratory test (**duration 02/03 hours**) after completion of all the experiments shall be conducted for 50 marks and scaled down to **10 marks**.
- Scaled-down marks of write-up evaluations and tests added will be CIE marks for the laboratory component of IPCC for **25 marks**.
- The student has to secure 40% of 25 marks to qualify in the CIE of the practical component of the IPCC.

Suggested Learning Resources:

Books

1. Control Systems Engineering, I J Nagrath, M. Gopal, New age international Publishers, Fifth edition.

Instructions to the Candidates

General Lab Guidelines:

- Conduct yourself in a responsible manner at all times in the laboratory. Intentional misconduct will lead to the exclusion from the lab.
- Do not wander around, or distract other students, or interfere with the laboratory experiments of other students.
- Read the handout and procedures before starting the experiments. Follow all written and verbal instructions carefully. If you do not understand the procedures, ask the instructor or teaching assistant.
- Attendance in all the labs is mandatory, absence permitted only with prior permission from Class teacher.
- The workplace has to be tidy before, during and after the experiment.
- Do not eat food, drink beverages or chew gum in the laboratory.
- Every student should know the location and operating procedures of all Safety equipment including First Aid Kit and Fire extinguisher.

DO'S: -

- Uniform and ID card are must.
- Strictly follow the procedures for conduction of experiments.
- Records have to be submitted every week for evaluation.
- Chairs and stools should be kept under the workbenches when not in use.
- After the lab session, switch off every supply, disconnect and disintegrate the experiments and return the components.
- Keep your belongings in designated area.
- Never use damaged instruments, wires or connectors. Hand these parts to the instructor/ teaching assistant.
- Sign the log book when you enter/leave the laboratory.

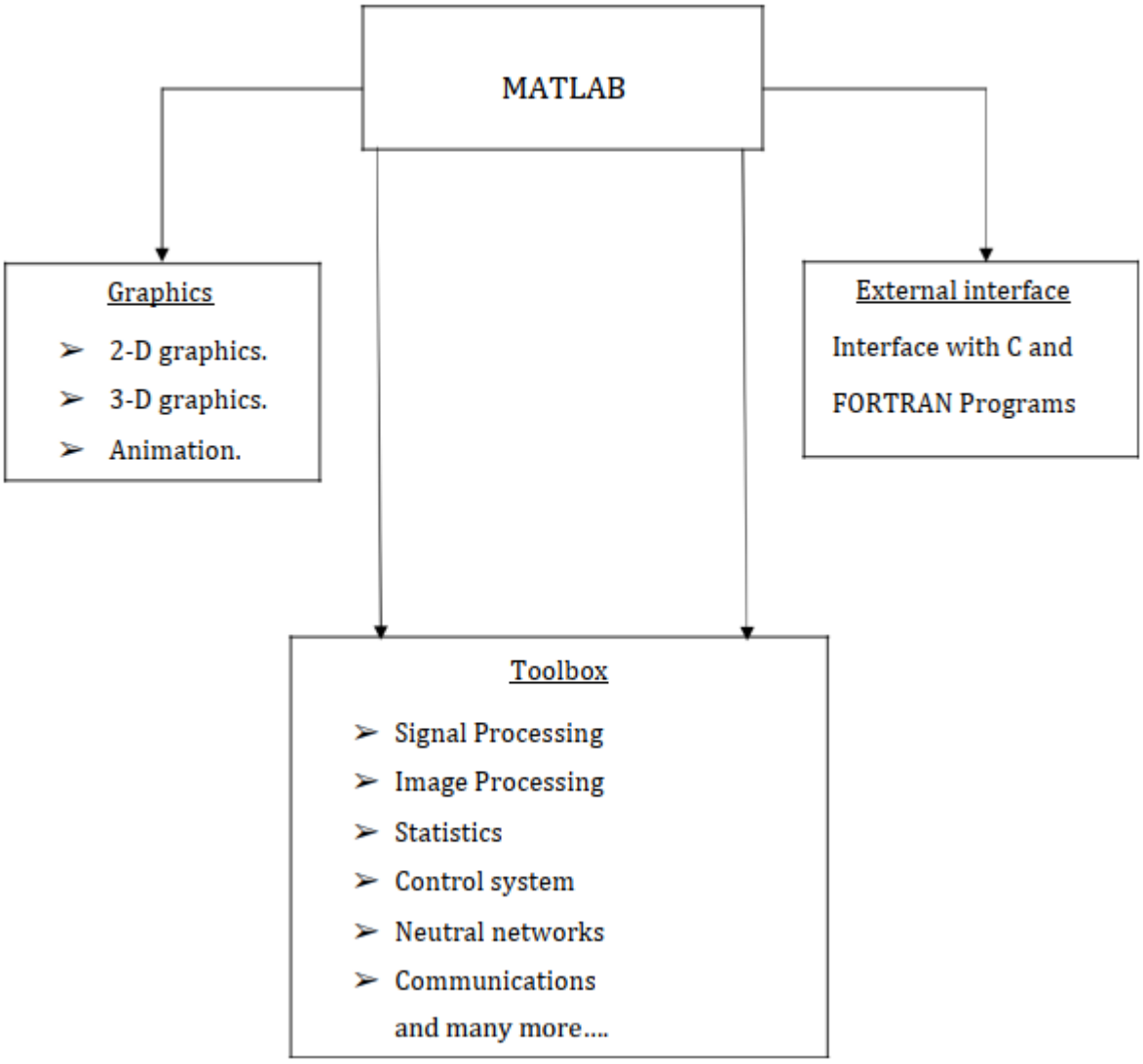
DONT'S: -

- Don't touch open wires unless you are sure that there is no voltage. Always disconnect the plug by pulling on the connector body not by the cable. Switch off the supply while you make changes to the experiment.
- Don't leave the experiment table unattended when the experimental setup supply is on.
- Students are not allowed to work in laboratory alone or without presence of the teaching staff/ instructor.
- No additional material should be carried by the students during regular labs.
- Avoid stepping on electrical wires or any other computer cables

INTRODUCTION TO MATLAB

MATLAB is a software package for high performance numerical computation and visualization provides an interactive environment with hundreds of built in functions for technical computation, graphics and animation. The MATLAB name stands for MATrix LABoratory.

The diagram shows the main features and capabilities of MATLAB.



At its core, MATLAB is essentially a set (a "toolbox") of routines (called "m files' or "mex files") that sit on your computer and a window that allows you to create new variables with names (e.g. voltage and time) and process those variables with any of those routines (e.g. plot voltage against time, find the largest voltage, etc.)

MATLAB® is a high-level language and interactive environment that enables you to perform computationally intensive tasks faster than with traditional programming languages such as C, C++, and FORTRAN.

Key Features:

- High-level language for technical computing.
- Development environment for managing code, files, and data.
- Interactive tools for iterative exploration, design, and problem solving.
- Mathematical functions for linear algebra, statistics, Fourier analysis,
- Filtering, optimization, and numerical integration.
- 2-D and 3-D graphics functions for visualizing data.
- Tools for building custom graphical user interfaces.
- Functions for integrating MATLAB based algorithms with external applications and languages, such as C, C++, FORTRAN, Java, COM, and Microsoft® Excel®.

MATLAB Windows:

MATLAB works with through three basic windows

Command Window: This is the main window. It is characterized by MATLAB command prompt (>>) when you launch the application program MATLAB puts you in this window all commands including those for user-written programs, are typed in this window at the MATLAB prompt.

Graphics window: The output of all graphics commands typed in the command window are flushed to the graphics or figure window, a separate gray window with white background color the user can create as many windows as the system memory will allow.

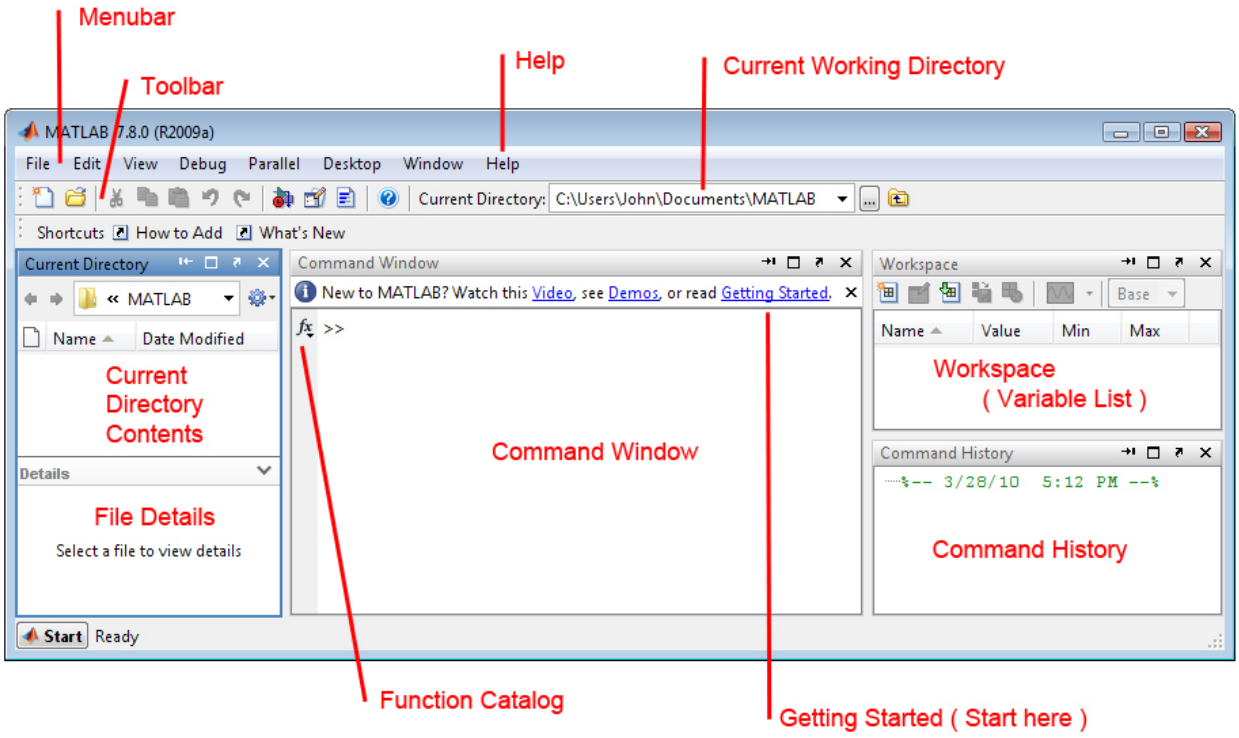
Edit window: This is where you write edit, create and save your own programs in files called M files.

Input-output: MATLAB supports interactive computation taking the input from the screen and flushing the output to the screen. In addition, it can read input files and write output files.

Data Type: The fundamental data-type in MATLAB is the array. It encompasses several distinct data objects- integers, real numbers, matrices. Character strings, structures and cells. There is no need to declare variables as real or complex, MATLAB automatically sets the variable to be real.

Dimensioning: Dimensioning is automatic in MATLAB. No dimension statements are required for vectors or arrays. We can find the dimensions of an existing matrix or a vector with the size and length commands.

The MATLAB Work Environment



Basic Functions in MATLAB:

Function	Description
disp()	The values or the text printed within single quotes is displayed on the output screen
clear	To clear all variables
close all	To close all graphics window
clc	To clear the command window
help	Searches for a help topic.
exp(x)	To compute the exponential value of x to the base e
abs(x)	To compute the absolute value of x
sqrt(x)	To compute the square root of x
log(x)	To compute the logarithmic value of x to the base e
input	Displays prompts and waits for input.
axis	Sets axis limits.
grid	Displays gridlines.
plot	Generates xy plot.
stem	Creates stem plot.
print	Prints plot or saves plot to a file.
title	Puts text at top of plot.
xlabel	Adds text label to x-axis.
ylabel	Adds text label to y-axis.
legend	Legend placement by mouse.

Experiment No 1

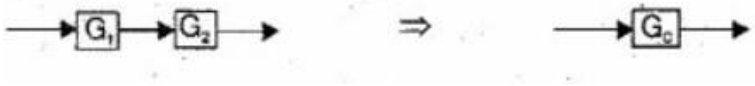
Implement Block diagram reduction technique to obtain transfer function a control system.

Aim: To write a MATLAB program to implement block diagram reduction technique to obtain transfer function a control system

Components Required: Computer System with MATLAB software

Theory:

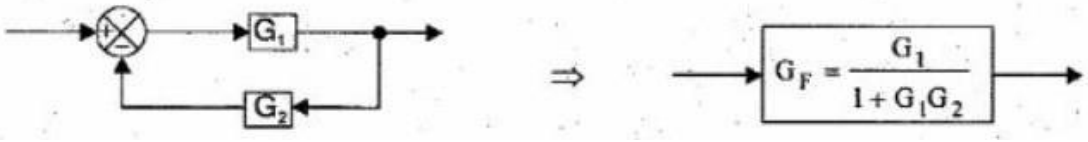
- When two blocks are connected in cascade, the resultant transfer function is the multiplication of the two blocks transfer function.



- When two blocks are connected in parallel, the resultant transfer function is the addition of the two blocks transfer function.



- When two blocks are connected in feedback, the resultant transfer function is as given below.



Consider the transfer functions of the two systems given below.

$$G_1(s) = \frac{8}{s^2+2s+9} \text{ and } G_2(s) = \frac{4}{s+6}$$

Program:

```
clc;
clear;
G1=tf('s');
G2=tf('s');
```

```
GC=tf('s');
GP=tf('s');
GF=tf('s');
num_cof1=[0 0 8];
den_cof1=[1 2 9];
disp('System 1');
G1=tf([num_cof1],[den_cof1])
num_cof2=[0 0 8];

den_cof2=[1 2 9];
disp('System 2');
G2=tf([num_cof2],[den_cof2])
[num_cofc,den_cofc]=series(num_cof1,den_cof1,num_cof2,den_cof2);
disp('Cascade System');
GC=tf([num_cofc],[den_cofc])
[num_cofp,den_cofp]=parallel(num_cof1,den_cof1,num_cof2,den_cof2);
disp('Parallel System');
GP=tf([num_cofp],[den_cofp])
[num_coff,den_coff]=feedback(num_cof1,den_cof1,num_cof2,den_cof2);
disp('Feedback System');
GF=tf([num_coff],[den_coff])
```

Expected Output:

```
System 1

G1 =

        8
  -------------
  s^2 + 2 s + 9

System 2

G2 =

        8
  -------------
  s^2 + 2 s + 9

Cascade System

GC =

                64
  --------------------------------
  s^4 + 4 s^3 + 22 s^2 + 36 s + 81

Parallel System

GP =

       16 s^2 + 32 s + 144
  --------------------------------
  s^4 + 4 s^3 + 22 s^2 + 36 s + 81

Feedback System

GF =

         8 s^2 + 16 s + 72
  --------------------------------
  s^4 + 4 s^3 + 22 s^2 + 36 s + 145
```

OBSERVATIONS AND OUTPUT

OBSERVATIONS AND OUTPUT

Experiment No 2

Implement Signal Flow graph to obtain transfer function a control system

Aim: To write and run the MATLAB coding to Implement Signal Flow graph to obtain transfer function a control system

Components Required: Computer System with MATLAB software

Theory:

The Mason's Gain Formula is a mathematical tool used in control system engineering to calculate the overall transfer function of a signal flow graph. The formula is given below

where,

N – total no. of forward paths

P_i – ith Forward path Gain

$$R(s) = \frac{\sum_{i=} P_i \Delta_i}{\Delta}$$

Δ **=1 – {***sum of all individual loop gains***} + {***sum of all non – touching loop gain products***}**

Δ_i – calculation of Δ for the non-touching part of the graph with ith forward path.

Program:

```
clc; clear;
% Define Gains G12=2;
G23=3;
G34=4;
G21=-0.5;
G43=-1;
% Forward Path Gain
p1=G12*G23*G34;
% Calculate Individual Loop Gain
p11=G12*G21;
p21=G34*G43;
% Calculation of Gain Products of two non touching loops
```

```
p12=p11*p21;
% Calculate Delta delta=1-
(p11+p21)+p12;
% Calculate del_i
del_i=1;
% Transfer Function
 disp('Transfer Function');
tf=(p1*del_i)/delta
```

Expected Output:

```
Transfer Function

tf =

     2.4000
```

OBSERVATIONS AND OUTPUT

Experiment No 3

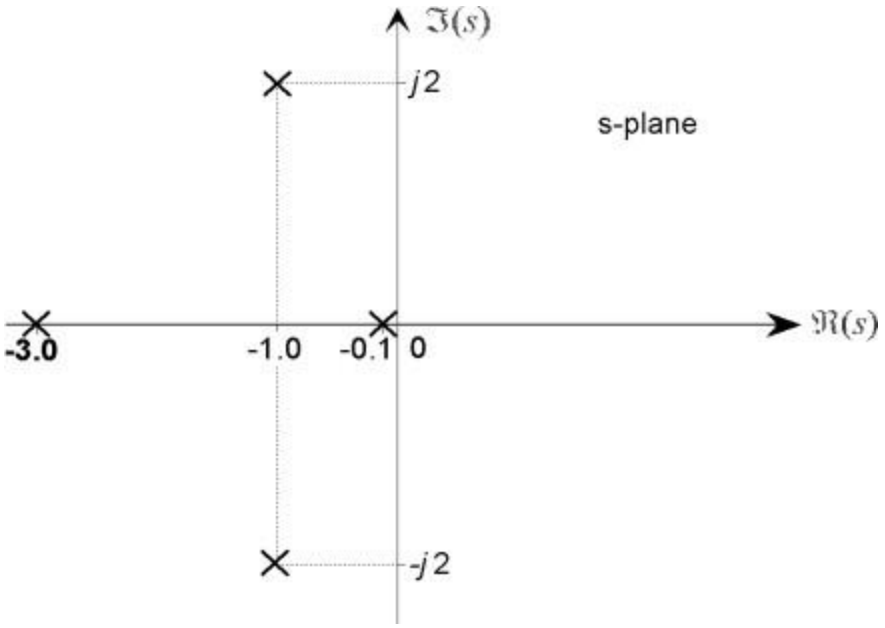
Simulation of poles and zeros of a transfer function

Aim: To write and run the MATLAB coding to Simulate the poles and zeros of a transfer function

Components Required: Computer System with MATLAB software

Theory:

A system is characterized by its poles and zeros in the sense that they allow reconstruction of the input/output differential equation. In general, the poles and zeros of a transfer function may be complex, and the system dynamics may be represented graphically by plotting their locations on the complex s-plane, whose axes represent the real and imaginary parts of the complex variable s. Such plots are known as pole-zero plots. It is usual to mark a zero location by a circle (○) and a pole location a cross (×). The location of the poles and zeros provide qualitative insights into the response characteristics of a system.



Program:

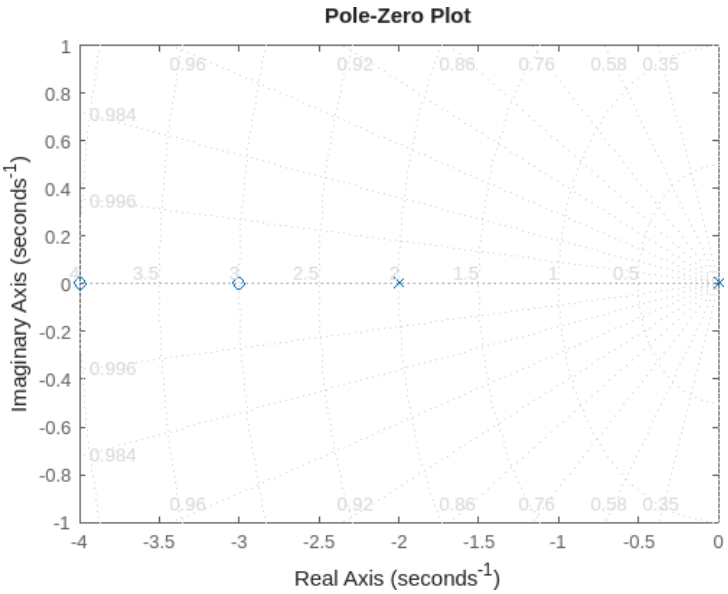
```
clc; clear;
  num=input('Enter the numerator coefficients');
  den=input('Enter the denominator cooefficients');
```

```
[b,a]=eqtflength(num,den);
[z,p,k]=tf2zp(b,a);
disp('Poles of the transfer function');
disp(p);
disp('zeros of the transfer function');
disp(z);
% Plot poles and zeros pzmap(p,z);
title('Pole-Zero Plot');
grid on;
```

Expected Output:

```
[1 7 12]
Enter the denominator cooefficients
[1 2]
Poles of the transfer function
     0
    -2

zeros of the transfer function
    -4
    -3
```



OBSERVATIONS AND OUTPUT

OBSERVATIONS AND OUTPUT

Experiment No 4

Implement time response specification of a second order Under damped System, for different damping factors.

Aim: **To write and run the MATLAB coding to implement time response specification of a second order Under Damped System, for different damping factors.**

Components Required: Computer System with MATLAB software

Theory:

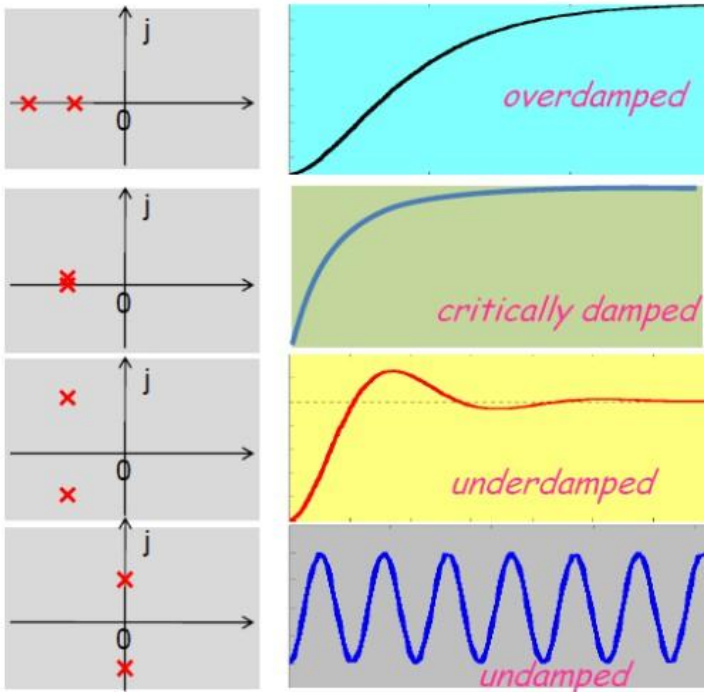
A second-order system: $G(s) = \dfrac{\omega_n^2}{s^2 + 2\zeta\omega_n s + \omega_n^2}$

$\zeta > 1$: $s_{1,2} = -\zeta\omega_n \pm \omega_n\sqrt{\zeta^2 - 1}$

$\zeta = 1$: $s_{1,2} = -\omega_n$

$0 < \zeta < 1$: $s_{1,2} = -\zeta\omega_n \pm j\omega_n\sqrt{1-\zeta^2}$

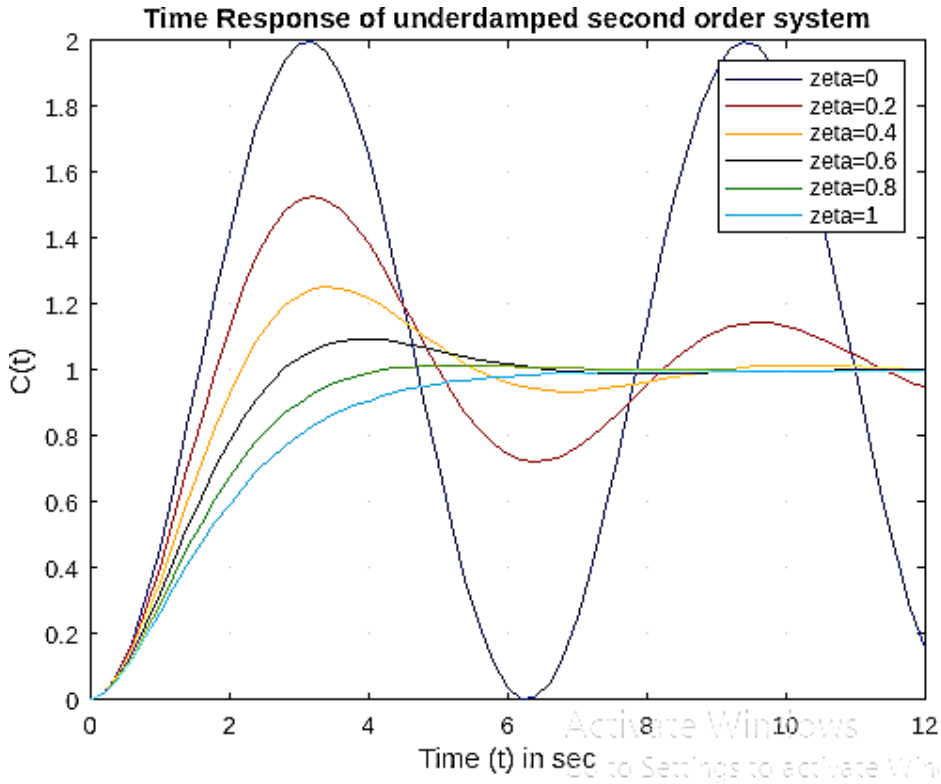
$\zeta = 0$: $s_{1,2} = \pm j\omega_n$



Program:

```
clc;  clear;
t=0:0.2:12;
c=zeros(61,6);
zeta=[0 0.2 0.4 0.6 0.8 1];
for n=1:6
      num_cof=[0 0 1];
      den_cof=[1 2*zeta(n) 1];
      c(1:61,n)=step(num_cof,den_cof,t);
end
plot(t,c); grid
on;
xlabel('Time (t) in sec');
ylabel('C(t)');
title('Time Response of underdamped second order system');
legend('zeta=0','zeta=0.2','zeta=0.4','zeta=0.6','zeta=0.8','zeta=1');
```

Expected Output:



Result:

OBSERVATIONS AND OUTPUT

OBSERVATIONS AND OUTPUT

Experiment No 5

Implement frequency response of a second order System.

Aim: To write and run the MATLAB coding to implement frequency response of a second order System.

Components Required: Computer System with MATLAB software

Theory:

Consider the transfer function of the second order closed loop control system as,

$$T(s) = \frac{C(s)}{R(s)} = \frac{\omega_n^2}{s^2 + 2\delta\omega_n s + \omega_n^2}$$

Substitute, $s = j\omega$ in the above equation.

$$T(j\omega) = \frac{\omega_n^2}{(j\omega)^2 + 2\delta\omega_n(j\omega) + \omega_n^2}$$

$$\Rightarrow T(j\omega) = \frac{\omega_n^2}{-\omega^2 + 2j\delta\omega\omega_n + \omega_n^2} = \frac{\omega_n^2}{\omega_n^2\left(1 - \frac{\omega^2}{\omega_n^2} + \frac{2j\delta\omega}{\omega_n}\right)}$$

$$\Rightarrow T(j\omega) = \frac{1}{\left(1 - \frac{\omega^2}{\omega_n^2}\right) + j\left(\frac{2\delta\omega}{\omega_n}\right)}$$

Let, $\frac{\omega}{\omega_n} = u$ Substitute this value in the above equation.

$$T(j\omega) = \frac{1}{(1-u^2) + j(2\delta u)}$$

Magnitude of $T(j\omega)$ is -

$$M = |T(j\omega)| = \frac{1}{\sqrt{(1-u^2)^2 + (2\delta u)^2}}$$

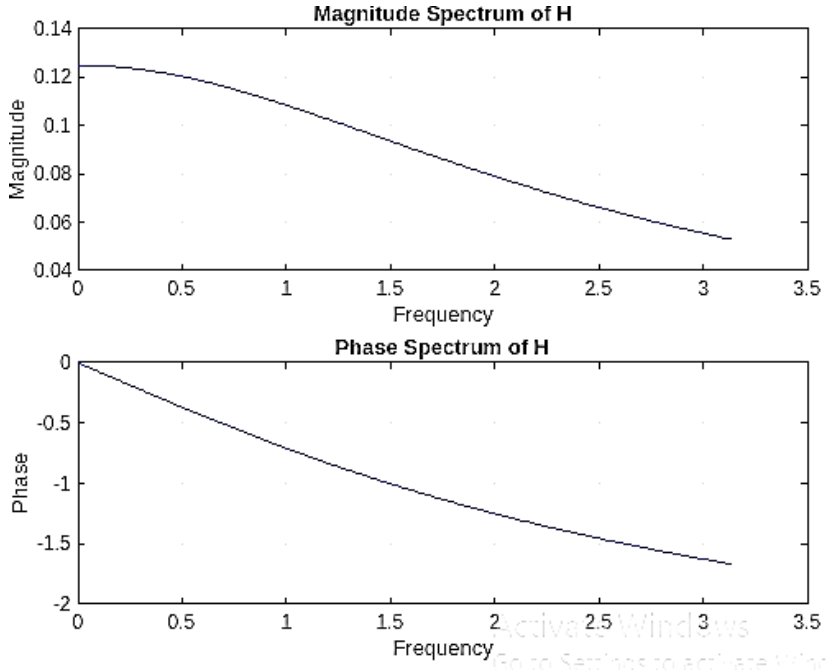
Phase of $T(j\omega)$ is -

$$\angle T(j\omega) = -tan^{-1}\left(\frac{2\delta u}{1-u^2}\right)$$

Program:

```
clc; clear;
  num1=[0 0 1];
  den1=[1 6 8];
  w=0:0.01:pi;
  h=freqs(num1,den1,w);
  mag=abs(h);
  phase=angle(h);
  figure;
  subplot(2,1,1);
  plot(w,mag);
  xlabel('Frequency');
  ylabel('Magnitude');
  title('Magnitude Spectrum of H');
  grid on;
  subplot(2,1,2);
  plot(w,phase);
  xlabel('Frequency');
  ylabel('Phase');
  title('Phase Spectrum of H');
  grid on;
```

Expected Output:



Result:

OBSERVATIONS AND OUTPUT

OBSERVATIONS AND OUTPUT

Experiment No 6

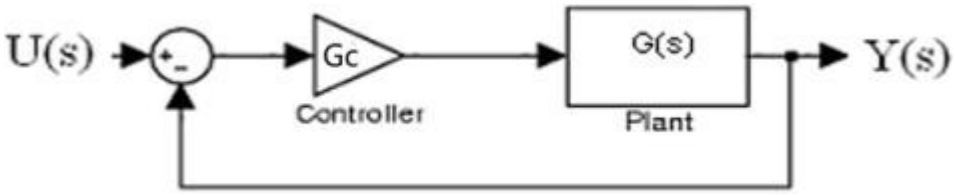
Implement frequency response of a lead lag compensator.

Aim: To write and run the MATLAB coding to implement frequency response of a lead lag compensator.

Components Required: Computer System with MATLAB software

Theory:

- Lead–lag compensators (e.g. PID controllers, etc.) influence disciplines as varied as robotics, satellite control, automobile diagnostics, and laser frequency stabilization.



- Both lead compensators and lag compensators introduce a pole-zero pair into the open loop transfer function, which can be written in the Laplace domain as

$$\frac{Y(s)}{X(s)} = \frac{s-z}{s-p}$$

where X is the input to the compensator, Y is the output, sis the complex Laplace transform variable, z is the zero frequency and p is the pole frequency. The pole and zero are both typically negative, or left of the zero in the complex plane. In a lead compensator, $|z| < |p|$, while in a lag compensator $|z| > |p|$.

- A lead-lag compensator consists of a lead compensator cascaded with a lag compensator. The overall transfer function can be written as

$$G_c(s) = \frac{Y_c(s)}{X_c(s)} = A \cdot \frac{(s-z_1)(s-z_2)}{(s-p_1)(s-p_2)}$$

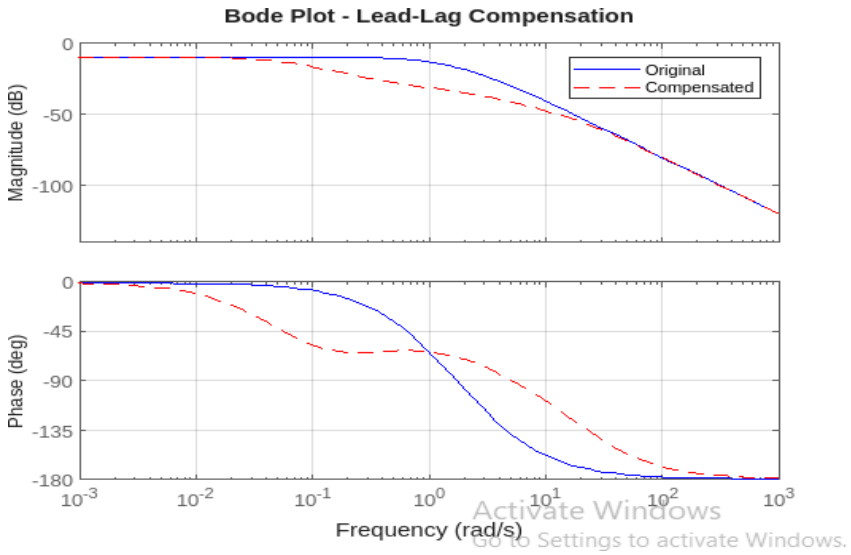
Typically, $|p1| > |z1| > |z2| > |p2|$, where $z1$ and $p1$ are the zero and pole of the lead compensator, $z2$ and $p2$ are the zero and pole of the lag compensator. The lead compensator provides phase lead at high frequencies. This shifts the root locus to the left, which enhances the responsiveness and stability of the system. The lag compensator provides phase lag at low

frequencies which reduces the steady state error.

Program:

```
clc;
clear;
% Define plant transfer function: G(s) = 1 / (s^2 + 4s + 3)
 G = tf(1, [1 4 3]);
% Lead compensator: D_lead(s) = (T1*s + 1) / (alpha*T1*s + 1)
 T1 = 0.5;
alpha = 0.1;
Lead = tf([T1 1], [alpha*T1 1]);
% Lag compensator: D_lag(s) = (T2*s + 1) / (beta*T2*s + 1)
 T2 = 2;
beta = 10;
Lag = tf([T2 1], [beta*T2 1]);
% Full compensator
Comp = Lead * Lag;
% Compensated system Gc
= Comp * G;
% Bode plot
bode(G, 'b', Gc, 'r--');
legend('Original', 'Compensated');
title('Bode Plot - Lead-Lag Compensation');
grid on;
```

Expected Output:



Activate Windows
Go to Settings to activate Windows.

Result:

OBSERVATIONS AND OUTPUT

OBSERVATIONS AND OUTPUT

Experiment No 7

Analyze the stability of the given system using Routh stability criterion.

Aim: To write and run the MATLAB coding to analyze the stability of the given system using Routh stability criterion.

Components Required: Computer System with MATLAB software

Theory:

Routh-Hurwitz Stability Criterion

Routh-Hurwitz stability criterion is having one necessary condition and one sufficient condition for stability. If any control system doesn't satisfy the necessary condition, then we can say that the control system is unstable. But, if the control system satisfies the necessary condition, then it may or may not be stable. So, the sufficient condition is helpful for knowing whether the control system is stable or not.

Necessary Condition for Routh-Hurwitz Stability

The necessary condition is that the coefficients of the characteristic polynomial should be positive. This implies that all the roots of the characteristic equation should have negative real parts.

Consider the characteristic equation of the order 'n' is –

$$a_0 s^n + a_1 s^{n-1} + a_2 s^{n-2} + \ldots + a_{n-1} s^1 + a_n s^0 = 0$$

Note that, there should not be any term missing in the nth order characteristic equation. This means that the nth order characteristic equation should not have any coefficient that is of zero value.

Program:

```
clc;
clear;
% Input the characteristic polynomial coefficients
% Example: s^4 + 3s^3 + 5s^2 + 2s + 4 a = [1 3 5 2 4];
n = length(a);
m = ceil(n/2);
% Initialize Routh table
Routh = zeros(n, m);
% Fill the first two rows
 Routh(1,:) = a(1:2:end);          % First row: even powers
Routh(2,1:length(a(2:2:end))) = a(2:2:end);          % Second row: odd powers
```

```
% Compute remaining rows


for i = 3:n
            for j = 1:m-1
                  if Routh(i-1,1) == 0
                     Routh(i-1,1) = 1e-6;     % Avoid division by zero
            end
Routh(i,j)=(Routh(i-1,1)*Routh(i-2,j+1)-Routh(i-2,1)*Routh(i-1,j+1))/Routh(i-1,1);
    end
end

% Display Routh table
disp('Routh-Hurwitz Table:');
disp(Routh);
% Check stability (count sign changes in first column)
first_col = Routh(:,1);
sign_changes = sum(sign(first_col(1:end-1)) ~= sign(first_col(2:end)));
if sign_changes == 0
disp('System is stable.');
 else
fprintf('System is unstable. Number of right-half s-plane poles: %d\n', sign_changes);
end
```

Result:

OBSERVATIONS AND OUTPUT

OBSERVATIONS AND OUTPUT

Experiment No 8

Analyze the stability of the given system using Root locus.

Aim: To write and run the MATLAB coding to analyze the stability of the given system using Root locus.

Components Required: Computer System with MATLAB software

Theory:

- Root locus is a graphical presentation of the closed-loop poles as a system parameter k is varied.
- The graph of all possible roots of this equation (K is the variable parameter) is called the root locus.
- The root locus gives information about the stability and transient response of feedback control systems.
- Properties and general rules for construction of the Root Loci is as follows:

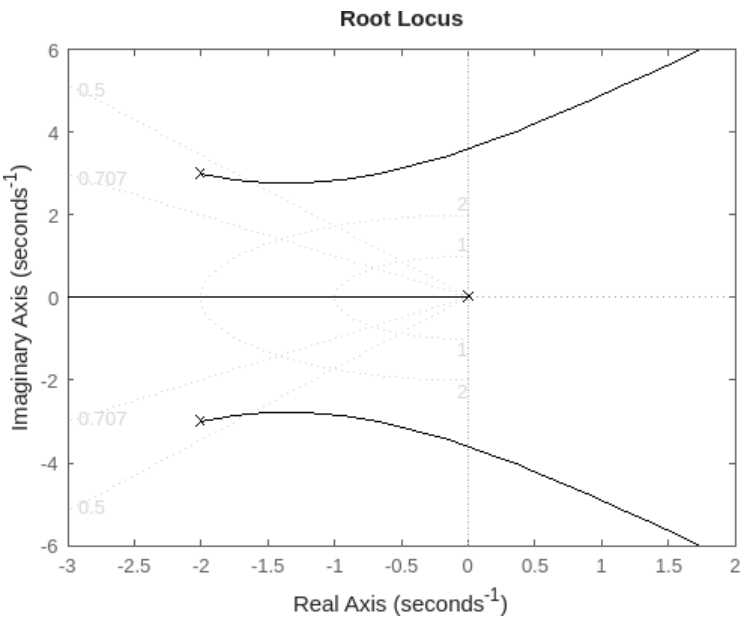
Rule no	Statement	Comments
Rule 1	The root locus is symmetric about the real axis	Root locus that we construct will always be symmetric about the real axis (irrespective of system)
Rule 2	Total number of loci	The total number of loci will be equal to max(p,z) where p is no: of the open loop poles and z is number of open loop zeros
Rule 3	Real axis loci	A point that lies on real axis basically lies on the root locus only if the total number of real open loop poles and open loop zeros present in the RHS of this point is odd
Rule 4	Angle of asymptotes	Total number of branches of the root locus tending towards infinity is equal to p – z. The angle of asymptotes gives us the direction along which these p-z branches approach infinity
Rule 5	centroid	Centroid is a point on real axis , through which the asymptotes pass.

Rule 6	Breakaway point	A break away point on the root locus is a point where the two poles will meet. Once they meet, they divide (split) i.e break away from the real axis.
Rule 7	Angle of departure / arrival	This gives us angles along which complex poles will depart(and complex zeros arrive) from their original position
Rule 8	Intersection with the imaginary axis	This gives us points on imaginary axis which the given root locus cut through while moving to right half of the s plane

Program:

```
clc; clear;
s=tf('s');
disp('The given transfer function is'); Gs=1/(s*(s^2+4*s+13))
rlocus(Gs,'k');
axis([-3 2 -6 6]);
sgrid([0.5,0.707],[90.5,1,2]);
```

Expected Output:



Result:

OBSERVATIONS AND OUTPUT

OBSERVATIONS AND OUTPUT

Experiment No 9

Analyze the stability of the given system using Bode plots.

Aim: To write and run the MATLAB coding to analyze the stability of the given system using Bode plots.

Components Required: Computer System with MATLAB software

Theory:

To analyze the stability of a system using a Bode plot, you examine the gain and phase margins. A stable system is characterized by both positive gain and phase margins, indicating a sufficient buffer before the system becomes unstable. The gain margin (GM) represents the amount the system's gain can increase before instability is reached, while the phase margin (PM) represents the amount of additional phase lag the system can tolerate before instability. *Steps to perform stability analysis using Bode plots:*

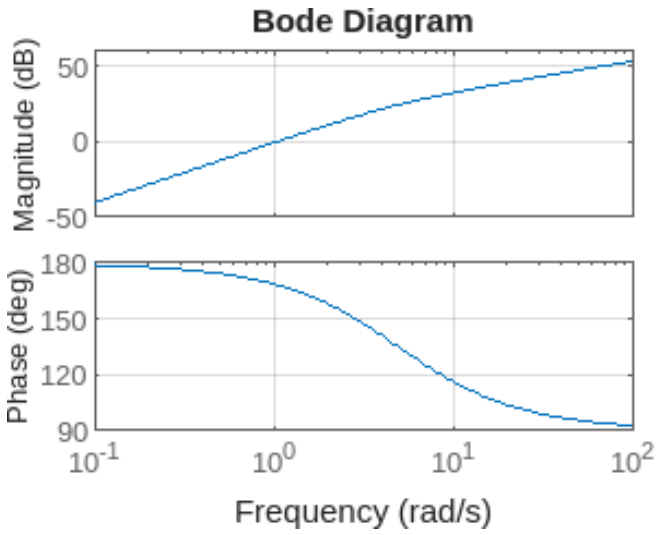
1. Obtain the Bode plot:
    - A Bode plot is a graphical representation of the frequency response of a system, consisting of two plots: magnitude (in dB) and phase (in degrees) as a function of frequency.
2. Identify Gain Crossover Frequency (ωgc):
    - This is the frequency at which the magnitude plot intersects the 0 dB line.
3. Identify Phase Crossover Frequency (ωpc):
    - This is the frequency at which the phase plot intersects the -180° line.
4. Determine Gain Margin (GM):
    - GM is the magnitude of the gain at the phase crossover frequency, expressed in dB, where the magnitude is measured at the phase crossover frequency.
    - A positive GM indicates stability.
    - A negative GM or a GM of zero or less indicates instability.
5. Determine Phase Margin (PM):
    - PM is the difference between 180° and the phase at the gain crossover frequency, expressed in degrees.
    - A positive PM indicates stability.
    - A negative PM or a PM of zero or less indicates instability.

Program:

```
clc; clear;
s=tf('s');
disp('The given transfer function is'); Gs=(s^2)/((1+0.2*s)*(1+0.02*2))
```

```
w=logspace(-1,2,200);
bode(Gs,w);
grid;
```

Expected Output:



OBSERVATIONS AND OUTPUT

OBSERVATIONS AND OUTPUT

Experiment No 10

Analyze the stability of the given system using Nyquist plot.

Aim: To write and run the MATLAB coding to Analyze the stability of the given system using Nyquist plot.

Components Required: Computer System with MATLAB software

Theory:

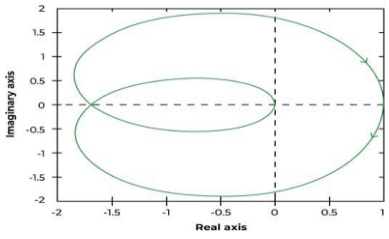
To analyse the stability of a system using a Nyquist plot, one needs to examine how the plot of the open-loop transfer function's frequency response (the Nyquist plot) encircles the critical point (-1, 0) in the complex plane. The Nyquist stability criterion states that the number of clockwise encirclements of the critical point by the Nyquist plot, minus the number of open-loop poles in the right-half s-plane, equals the number of closed-loop poles in the right-half s-plane. If the number of closed-loop poles in the right-half s-plane is zero, the system is stable.

1. ***Plotting the Nyquist Diagram:***

- Calculate the open-loop transfer function, GH(s), where G(s) is the system's transfer function and H(s) is the feedback transfer function.
- Substitute $s = j\omega$ (where ω is the frequency) into GH(s) to get GH($j\omega$).
- Plot the real and imaginary parts of GH($j\omega$) for different values of ω, forming the Nyquist plot.

2. ***Applying the Nyquist Stability Criterion:***

- Identify the number of open-loop poles in the right-half s-plane (P): This is determined by analysing the open-loop transfer function GH(s).
- Count the number of clockwise encirclements of the critical point (-1, 0) by the Nyquist plot (N): If the plot encircles the point in the clockwise direction, count it as one encirclement.
- Apply the Nyquist criterion: The number of closed-loop poles in the right-half s-plane (Z) is given by $Z = N + P$.
- Determine stability:
  - If $Z = 0$, the system is stable.
  - If $Z > 0$, the system is unstable.

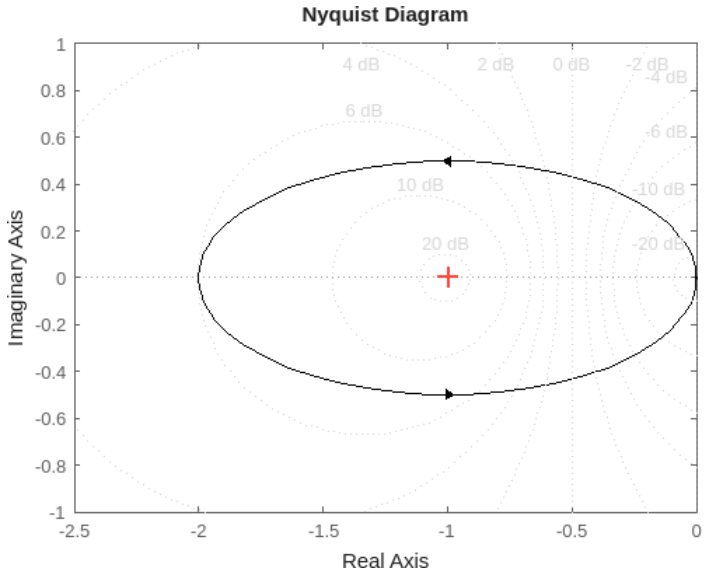


Program:

```
clc;
clear;
```

```
s=tf('s');
disp('The given transfer function is');
Gs=(s+2)/((s+1)*(s-1))
nyquist(Gs,'k');
axis([-2.5 0. -1 1]);
grid;
```

Expected Output:



OBSERVATIONS AND OUTPUT

OBSERVATIONS AND OUTPUT

Experiment No 11

Obtain the time response from state model of a system.

Aim: To write and run the MATLAB coding to obtain the time response from state model of a system.

Components Required: Computer System with MATLAB software

Theory:

- State-space models are models that use state variables to describe a system by a set of first-order differential or difference equations, rather than by one or more nth-order differential or difference equations.
- State variables x(t) can be reconstructed from the measured input-output data, but are not themselves measured during an experiment.
- The state-space model structure is a good choice for quick estimation because it requires you to specify only one input, the model order, n.
- The model order is an integer equal to the dimension of x(t) and relates to, but is not necessarily equal to, the number of delayed inputs and outputs used in the corresponding linear difference equation.
- A state-space representation is a mathematical model of a physical system as a set of input, output and state variables related by first-order differential equations. "State space" refers to the space whose axes are the state variables.
- The state of the system can be represented as a vector within that space.
- A state-space model object representing the continuous-time state-space model.

$$\dot{x} = Ax(t) + Bu(t)$$

$$y = Cx(t) + Du(t)$$

Program:

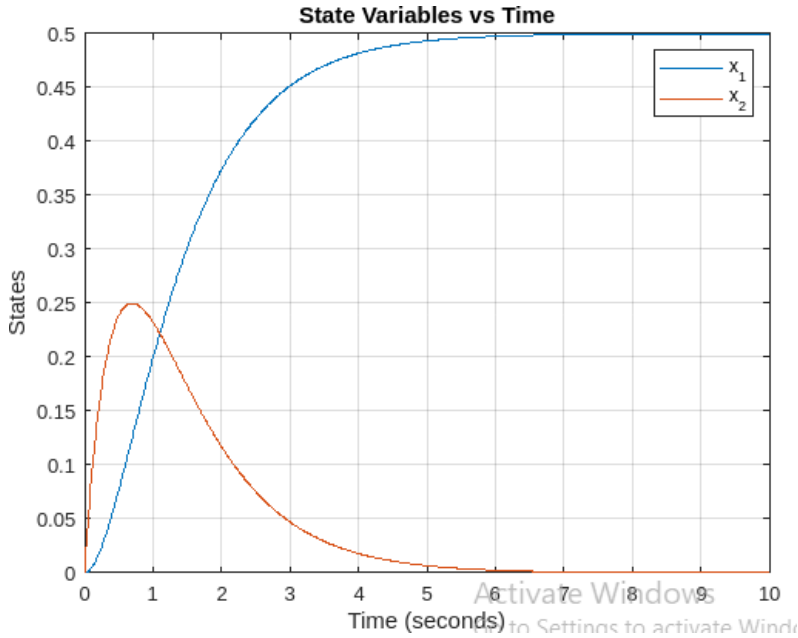
```
clc;
clear;
% Define the state-space model
% Example system:
A = [0 1; -2 -3];
B = [0; 1];
C = [1 0];
D = 0;
% Create state-space system
sys = ss(A, B, C, D);
% Time vector

t = 0:0.01:10;
% Step response
```

```
[y, t, x] = step(sys, t);

% Plot output response figure;
plot(t, y, 'LineWidth', 2);
grid on;
title('Step Response from State-Space Model'); xlabel('Time
(seconds)');
ylabel('Output');
% Optional: Plot state variables figure;
plot(t, x); grid
on;
title('State Variables vs Time'); xlabel('Time
(seconds)');
ylabel('States');
legend('x_1', 'x_2');
```

Expected Output:



Result:

OBSERVATIONS AND OUTPUT

OBSERVATIONS AND OUTPUT

Experiment No 12

Implement PI and PD Controllers.

Aim: To write and run the MATLAB coding to Implement PI and PD Controllers.

Components Required: Computer System with MATLAB software

Theory:

PD Controller: **D mode is used when prediction of the error can improve control or when it necessary to stabilize the system. From the frequency characteristic of D element, it can be seen that it has phase lead of 90°. Often derivative is not taken from the error signal but from the system output variable. This is done to avoid effects of the sudden change of the reference input that will cause sudden change in the value of error signal. Sudden change in error signal will cause sudden change in control output. To avoid that it is suitable to design D mode to be proportional to the change of the output variable.**

$$c(t) = T_d \frac{de}{dt}$$

where

- $c(t)$ is the controller output
- T_d is the derivative time constant
- de is the differential change in error
- dt is the differential change in time

PI Controller**: PI controller will eliminate forced oscillations and steady state error resulting in operation of on-off controller and P controller respectively. However, introducing integral mode has a negative effect on speed of the response and overall stability of the system. Thus, PI controller will not increase the speed of response. PI-control correlates the controller output to the error and the integral of the error.**

$$c(t) = \frac{1}{T_i} \int e(t)dt + c\,(t_0)$$

where

- $c(t)$ is the controller output,
- K_c is the controller gain,
- T_i is the integral time,
- $e(t)$ is the error, and
- C is the initial value of controller

Program:

```
clear; close
all;
% Define the plant
% Example: G(s) = 1 / (s^2 + 5s + 6)
num = 1;
den = [1 5 6];
G = tf(num, den);
% ----- PI Controller -----
Kp_PI = 2;
Ki = 1;
PI = tf([Kp_PI Ki], [1 0]);  % PI(s) = Kp + Ki/s
% Open-loop with PI OL_PI
= series(PI, G);
% Closed-loop with PI
CL_PI = feedback(OL_PI, 1);
% ----- PD Controller -----
Kp_PD = 2;
Kd = 1;
PD = tf([Kd Kp_PD], 1);  % PD(s) = Kp + Kd*s
% Open-loop with PD OL_PD
= series(PD, G);
% Closed-loop with PD
CL_PD = feedback(OL_PD, 1);
% ----- Plot Step Responses ----- t
= 0:0.01:10;
figure; step(CL_PI,
t); hold on;
step(CL_PD, t); grid
on;
legend('PI Controller', 'PD Controller'); title('Step
Response with PI and PD Controllers'); xlabel('Time
(seconds)');
ylabel('Output');
```

Expected Output:



Activate Windows
Go to Settings to activate Windc

Result:

OBSERVATIONS AND OUTPUT

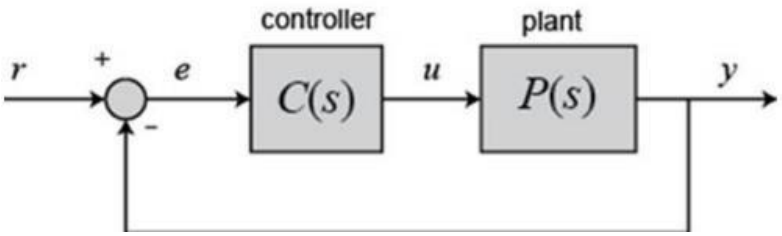
OBSERVATIONS AND OUTPUT

Experiment No 13

Implement a PID Controller and hence realize an Error Detector

Aim: To write and run the MATLAB coding to Implement a PID Controller and hence realize an Error Detector.
Components Required: Computer System with MATLAB software

Theory:
Consider the following unity feedback system:



The transfer function of a PID controller is found as:

$$K_p + \frac{K_i}{s} + K_d s = \frac{K_d s^2 + K_p s + K_i}{s}$$

K_p = Proportional gain,

K_i = Integral gain,

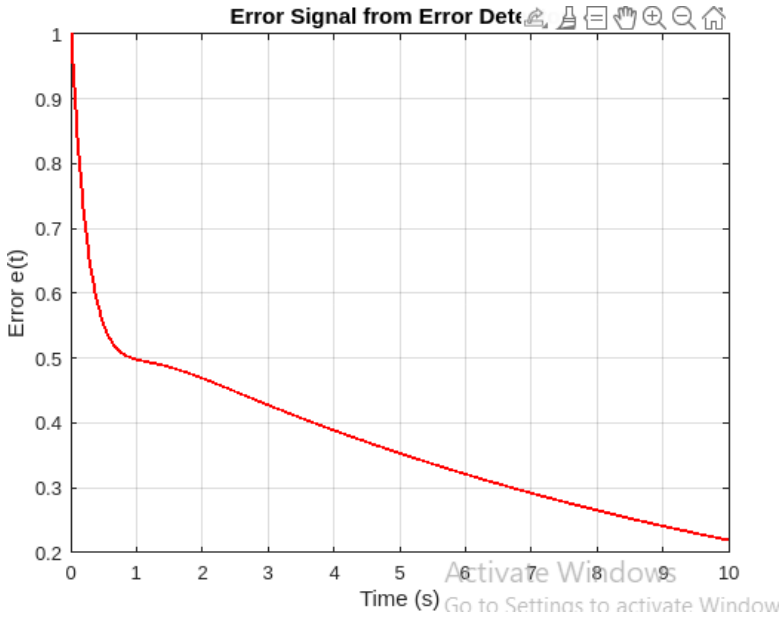
K_d = Derivative gain

Program:

```
clc; clear;
close all;
  % --- Define the Plant ---
  % Example: G(s) = 1 / (s^2 + 4s + 6)
  num = 1;
  den = [1 4 6];
  G = tf(num, den);
  % --- PID Controller ---
  Kp = 5;
  Ki = 1;
  Kd = 2;
  % PID(s) = Kp + Ki/s + Kd*s
  PID = Kp + tf([Ki], [1 0]) + tf([Kd 0], 1);  % PID controller
  % --- Open-loop System --- open_loop
  = series(PID, G);
  % --- Closed-loop System with Unity Feedback ---
  closed_loop = feedback(open_loop, 1);
```

```
% --- Time Vector and Step Input ---
t = 0:0.01:10;
r = ones(size(t));  % Unit step input (reference)
% --- Simulate System Response ---
[y, t_out] = step(closed_loop, t);  % System output
% --- Error Detector: e(t) = r(t) - y(t) ---
e = r - y';  % Transpose y to match dimensions
% --- Plot Output Response --- figure;
plot(t_out, y, 'b', 'LineWidth', 2); hold on;
plot(t_out, r, 'k--');
grid on;
xlabel('Time (s)');
ylabel('Output');
title('System Output vs Reference Input');
legend('System Output', 'Reference Input');
% --- Plot Error Signal ---
figure;
plot(t_out, e, 'r', 'LineWidth', 2);
grid on;
xlabel('Time (s)');
ylabel('Error e(t)');
title('Error Signal from Error Detector');
```

Expected Output:



OBSERVATIONS AND OUTPUT

OBSERVATIONS AND OUTPUT

OBSERVATIONS AND OUTPUT